PhysiTrust-FL: Grid-Challenged Proof-of-Presence and Hardware-Bound Measurement Provenance for Trust Management in Federated Grid-Edge Anomaly Detection

Manojkumar Ashokkumar (23BKT0135)
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
[your VIT email]@vitstudent.ac.in

***Abstract*—Federated learning lets smart-grid edge devices train a shared anomaly detector without sending raw telemetry, but the aggregator has no way to know whether an update really comes from a device that is installed on the feeder it claims. Signatures and decentralized identifiers prove who signed an update, not where the signer is or whether its training data was ever measured. This paper proposes PhysiTrust-FL, a Web3 public key infrastructure for federated grid-edge anomaly detection that adds two physical checks to the usual identity check. First, a grid-challenged proof-of-presence: in every round the utility commits to a secret seed, smart inverters inject a small pseudo-random reactive power dither derived from it, and each node must return a voltage and branch-flow response whose fingerprint matches its registered bus. Second, hardware-bound provenance: a secure element signs every raw measurement window, and the verifier spot-checks signed windows against the submitted model. The results drive an on-chain trust and key-state machine and a trust-weighted median aggregator. On the IEEE 33-bus feeder, off-grid and replayed devices were never accepted, honest nodes were accepted 99.9% of the time with a 30 kvar dither that moves bus voltage by less than 0.24%, and the global detector kept an F1 score above 0.86 with 60% of nodes sending sign-flipped updates or with 64 credentialed Sybil devices, where FedAvg, Multi-Krum, median and trimmed mean failed.**

***Index Terms*—Federated learning, public key infrastructure, decentralized identifiers, trust management, smart grid, anomaly detection, physical-layer authentication, secure element**

I. Introduction

Distribution feeders now carry many measuring devices: smart meters, feeder-section monitors and inverter controllers. Each of them sees a small part of the grid, and anomalies such as false data injection (FDI), frozen sensors or tampered meter readings are easier to catch when the devices learn from each other [1], [2], [3]. Federated learning (FL) [4] fits this setting well, because only model updates leave the device and the raw telemetry stays local.

The weak point is trust. A plain FL server accepts whatever arrives. Byzantine-robust rules such as Krum, the coordinate-wise median and the trimmed mean [5], [6] assume that honest clients are the majority, and that assumption is broken by attacks that are tuned against the rule [7] or by an attacker who simply adds identities. Blockchain-based FL and decentralized identifiers (DIDs) [8], [9], [10] fix part of the problem: every update is signed, the signer is looked up in an on-chain registry, and the model hash is anchored. Granted patents already cover smart-contract coordination of FL rounds, reputation-weighted aggregation and DID-gated device updates [11], [12], [13]. What none of these designs can tell is whether a credentialed device is really installed at the point of the grid it claims, and whether the data it trained on was measured at all. A meter bought, credentialed and then kept on a lab bench signs updates just as well as one on the feeder.

A power network has a property that ordinary IoT networks lack: the utility can change the physical state of the grid on purpose and in a controlled way, and every device that is really connected sees the change according to its electrical position. Physical watermarking [14], [15] and grid-frequency signatures [16], [17] have used such signals to detect FDI or to timestamp media. This paper uses them as a credential.

The contributions are as follows.

- A grid-challenged proof-of-presence (PoP). A seed committed on the ledger drives pseudo-random reactive power dither from several smart inverters. Each node's measured voltage and branch-flow response forms a fingerprint that is tested against the utility model of its registered bus. The seed is revealed only after the nodes commit their measurements, so the response cannot be precomputed.
- Hardware-bound measurement provenance. A secure element in the metering path signs every raw window, the training set is committed as a Merkle root inside the signed update, and the verifier spot-checks revealed windows against the submitted model.
- A DID-bound trust and key-state machine (active, quarantined, revoked) driven by both physical checks and by update geometry, together with a trust-weighted median aggregator.
- A full simulation on the IEEE 33-bus feeder with real ECDSA signatures, comparing five aggregation baselines under three poisoning attacks, Sybil floods, relocation and replay, with ablation and overhead analysis.

Section II reviews related work. Section III gives the system and threat model. Section IV describes the scheme. Sections V and VI present the setup and results, Section VII discusses the novelty and limits, and Section VIII concludes.

II. Related Work

FL for grid anomaly detection. Jithish et al. [1] trained anomaly detectors on smart-meter data with FL and showed accuracy close to centralized training. Li et al. [2] used secure federated deep learning for FDI detection across utilities. These works assume honest clients and do not authenticate the source of the data.

Robust aggregation. Krum [5], median and trimmed mean [6] filter updates statistically. FLTrust [18] scores every update against an update computed on a small clean root dataset held by the server. Fang et al. [7] showed that the first group can be broken by attacks crafted against the rule, and backdoors can pass all of them [19]. All of these act after the update is received and none of them limit how many identities an attacker can bring.

Blockchain and DID based FL. Blockchained FL [8] records updates on a ledger. TBFL [9] gates participants with DIDs and verifiable credentials and stops Sybils that lack a credential. Reputation-weighted blockchain FL appears in patents such as CN114491623B [12]. These methods prove identity, not physical location or data origin.

Physical-layer signatures. Dynamic watermarking and moving target defense inject known perturbations to expose FDI in grid control loops [14], [15]. Grid frequency and voltage signatures have been used to authenticate the time and place of media and IoT data [16], [17]. Frequency is almost identical over a whole interconnection, so it cannot tell feeders apart, and these methods are passive or are not linked to any key management. Table I compares the features.

TABLE I. Feature comparison with prior work

Work	Signed	Robust to stats	Physical presence	HW data	Key state
FedAvg, Krum, median [4], [5], [6]	No	Partly	No	No	No
FLTrust [18]	No	Yes	No	No	No
Blockchain FL [8], [11]	Yes	No	No	No	No
Reputation BFL [12]	Yes	Yes	No	No	Partly
TBFL (DID + VC) [9]	Yes	No	No	No	Revoke
Watermarking [14], [15]	No	n/a	Detects FDI	No	No
ENF / ENV auth. [16], [17]	Partly	n/a	Region	No	No
PhysiTrust-FL (this work)	Yes	Yes	Bus level	Yes	Auto

III. System and Threat Model

Entities. (i) The utility, which owns the feeder model, issues credentials and controls a few smart inverters. (ii) Edge nodes, one per bus, each with a metering front end, a secure element (SE) holding a P-256 key, and a wallet holding a secp256k1 key whose address is the node DID. (iii) A ledger running the registry, round, trust and anchor contracts. (iv) A verifier and aggregator whose decisions can be recomputed by anyone from on-chain data.

Assumptions. The SE key cannot be extracted, the utility feeder model is reasonably accurate, and the chain gives ordering of transactions. The utility is trusted for onboarding, which is already true for metering in practice.

Threats. T1 off-grid Sybil devices that hold valid credentials; T2 replay of an old signed window or update; T3 relocation of a genuine device or reuse of its identity at another bus; T4 software-fabricated telemetry; T5 model poisoning by compromised but genuine nodes (sign-flip, noise, adaptive data poisoning); T6 tampering with updates in transit. T6 and forged signatures are stopped by ECDSA alone and are not discussed further.

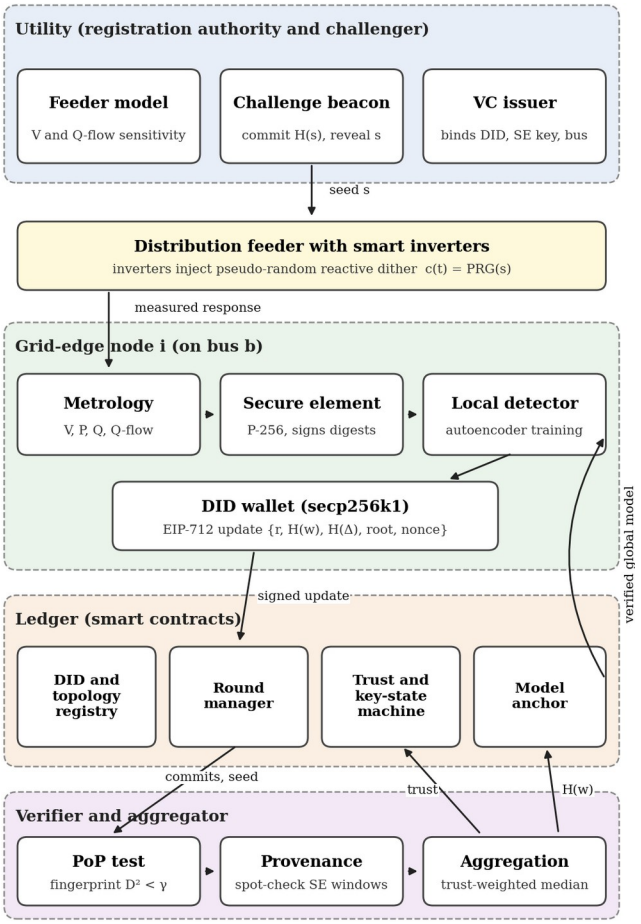


Fig. 1. PhysiTrust-FL architecture. The utility challenges the feeder, edge nodes return SE-signed measurements and EIP-712 signed updates, the ledger keeps identities, rounds, trust and model hashes.

IV. Proposed PhysiTrust-FL Scheme

A. Identity and Onboarding

Node i on bus b_i creates a secp256k1 wallet key whose address a_i serves as its DID in the did:ethr form [10], [20], and its secure element creates a P-256 key pair. The utility checks the installation and issues a credential that binds a_i, the hash of the SE public key and b_i. Only the credential hash and the bus index go on chain, in a registry that also stores the feeder topology. The key state starts as ACTIVE.

B. Round Protocol

Each FL round r follows the steps of Algorithm 1. The order of commit and reveal is the core of the protocol: a node must commit to what it measured before anyone outside the utility knows what it should have measured.

Algorithm 1: one PhysiTrust-FL round

1. Aggregator posts $H(w_r)$; utility posts $h = \text{keccak}(s_r)$ for a fresh secret seed s_r.
2. Inverters apply the dither $c_a(t)$ derived from s_r for L seconds.
3. Each SE signs its raw window; node commits the digest on chain.
4. After the commit deadline the utility reveals s_r; contract checks $\text{keccak}(s_r) = h$.
5. Node trains locally and submits the EIP-712 signed update (10).
6. Contract checks admission (11); verifier runs PoP (6) and provenance (9).
7. Trust (12), (13) and key states are updated on chain.
8. Trust-weighted median (14) gives w_{r+1}; $H(w_{r+1})$ is anchored; nodes verify it.

C. Grid-Challenged Proof-of-Presence

Let A inverters sit at buses with nominal reactive output $\bar{q}_a$. From the seed, each inverter gets a pseudo-random chip sequence at 1 Hz

$$c_a(t)=2\,bit_t\big(SHA256(s_r\|a)\big)-1 \quad (1)$$

$$q_a(t)=\bar{q}_a-\Delta q\,c_a(t) \quad (2)$$

where Δq is a small dither amplitude. Node *i* records its bus voltage magnitude and the reactive power flow on the branch that feeds its bus. For each series $y_i(t)$ we use

$$y_i(t)=\sum_{a=1}^{A} g_{i,a}\,c_a(t)+\beta_0+\beta_1 t+\beta_2 t^2+\varepsilon_i(t) \quad (3)$$

and estimate the gains by least squares, $\acute{\boldsymbol{g}}_i=(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}_i$ (4), with standard errors $\acute{\sigma}_{i,k}$ taken from the residuals. The quadratic trend absorbs slow load changes, and because the chips are white the standard errors stay valid even though load noise is strongly correlated in time. The verifier computes the expected gains from the feeder model at the loading seen at the substation. For the voltage magnitude V of bus b_i,

$$g^{V}_{i,a}=\frac{1}{2}\big[V_{b_i}(\bar{q}_a-\Delta q)-V_{b_i}(\bar{q}_a+\Delta q)\big] \quad (5)$$

and in the same way for branch flow. The 2*A* gains form the fingerprint of the bus, and the test statistic is

$$D_i^2=\sum_{k=1}^{2A}\frac{(\acute{g}_{i,k}-g_{i,k})^2}{\acute{\sigma}^2_{i,k}+\kappa^2 g^2_{i,k}} \quad (6)$$

where κ allows for model error. The node passes if $D_i^2<\gamma=\chi^2_{2A}(1-\alpha)$, which is 26.1 for $A = 4$ and $\alpha = 10^{-3}$. The two halves of the fingerprint carry different information. The flow gain is close to one when the inverter is downstream of the node and close to zero otherwise, so it encodes which lateral the node is on. The voltage gain grows with the reactance shared by the node and the inverter paths, so it encodes how far along the lateral the node sits. A device that is not on the feeder sees no dither, a replayed window carries the chips of an old seed, and a relocated device shows the fingerprint of the wrong bus.

D. Hardware-Bound Measurement Provenance

The secure element sits between the metering front end and the processor. For every raw window it computes

$$d_{i,k}=SHA256(\boldsymbol{x}_{i,k}\|n_{i,k}),\sigma_{i,k}=Sig(d_{i,k}) \quad (7)$$

where $n_{i,k}$ is a monotonic counter and Sig is the P-256 signature made inside the SE, so the processor cannot create a signed window that the hardware did not measure. The training set is committed as

$$R_i=Merkle(d_{i,1},\ldots,d_{i,n_i}) \quad (8)$$

and R_i is part of the signed update. In each round the verifier asks for *c* random windows $\mathscr{S}_i$, checks their SE signatures and applies a loss gate on the submitted model

$$\bar{\ell}(w_r+\Delta_i;\mathscr{S}_i)\le\rho\,\bar{\ell}(w_r;\mathscr{S}_i)+\epsilon \quad (9)$$

Since only SE-signed windows can be revealed, a poisoned model cannot be defended with windows that were made up to fit it. Revealed windows go only to the verifier run by the utility, which already reads these meters.

E. Signed Update and Admission

The update is signed with EIP-712 typed data [21], which binds the chain, the contract, the round and the model the node started from:

$$m_i=H_{712}(dom,a_i,r,H(w_r),H(\Delta_i),R_i,n_i) \quad (10)$$

$$\mathscr{A}_i=[\acute{a}_i=a_i]\wedge[st_i\neq R]\wedge[n_i\notin\mathscr{N}]\wedge[t_i^c<t^s] \quad (11)$$

Here $\acute{a}_i$ is the address returned by ecrecover on the signature, R the REVOKED state, $\mathscr{N}$ the set of used nonces and $t_i^c<t^s$ means the measurement commit came before the seed reveal. Signature, replay and late-commit failures are rejected before any learning check.

F. Trust Score and Key-State Machine

Let $\tilde{\Delta}$ be the coordinate-wise median of the updates that passed both physical checks. The round score and trust are

$$s_i=1[D_i^2<\gamma]\,1[prov_i]\,max\big(0,\cos(\Delta_i,\tilde{\Delta})\big) \quad (12)$$

$$T_i\leftarrow\lambda T_i+(1-\lambda)s_i \quad (13)$$

A node whose trust falls below τ is moved to QUARANTINED. It is still challenged but its updates are not aggregated. Two good rounds bring it back, while three rounds in quarantine set it to REVOKED on chain, after which a new key needs a new credential. A rotated key therefore does not get its old standing for free.

G. Aggregation

$$w_{r+1}=w_r+wmed\big(\{clip(\Delta_i),T_i\}_{i\in\mathscr{G}_r}\big) \quad (14)$$

where $\mathscr{G}_r$ are ACTIVE nodes that passed all checks, clip scales each update to the median norm, and wmed is the coordinate-wise median weighted by trust.

V. Experimental Setup

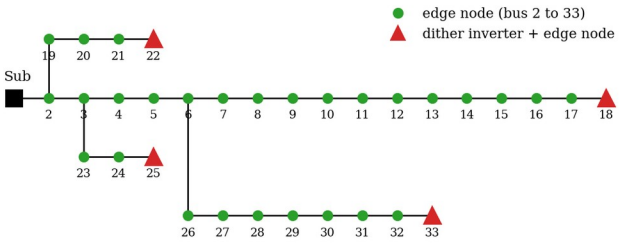


Fig. 2. IEEE 33-bus test feeder [22] with one edge node per load bus and dither inverters at the four lateral ends.

All experiments use the IEEE 33-bus, 12.66 kV feeder [22] shown in Fig. 2, solved with a backward/forward sweep power flow (base case: minimum voltage 0.913 p.u. at bus 18). For PoP windows the loads vary as a 1 Hz AR(1) process. For FL, six days of 1-minute data are produced for every bus with a daily load shape and random noise. Each node uses 10-minute windows of voltage, P, Q and upstream Q-flow (40 inputs). Days 1 to 4 are used for training, day 5

sets the alarm threshold (99th percentile of clean scores) and day 6 is the test set, where half of the windows carry one of six injected anomalies: bias FDI, ramp, scaling, frozen sensor, noise burst and a P and V move that breaks physics.

Attacks on compromised genuine nodes are sign-flip (update times −4), Gaussian noise (five times the honest norm) and adaptive data poisoning (ADP), where the node trains on 50% fabricated anomalous windows and then shrinks its update until its own loss stays within 1.15 times the previous one, so that a loss gate is likely to pass. Sybils hold valid credentials and copied data, run ADP, but are not on the feeder. Multi-Krum is told the true number of attackers, which favours it. FLTrust gets a 120-window clean root set from three buses. Signatures use eth-account (EIP-712, ecrecover) and the cryptography library (P-256). Contract logic is emulated in Python and gas is estimated from the EVM fee schedule. Table II lists the main settings.

TABLE II. SIMULATION PARAMETERS

Parameter	Value
Edge nodes / dither inverters	32 / 4 (buses 18, 22, 25, 33)
Dither Δq, window L	30 kvar per inverter, 120 s at 1 Hz
Noise: voltage / Q-flow	5×10^{-4} p.u. / 0.2%
Load noise (PoP)	AR(1), σ = 2%, φ = 0.95
κ, α, γ	0.02, 10^{-3}, 26.1
Autoencoder	40-12-40, tanh, SGD 0.05, batch 32
Rounds, local epochs, seeds	25, 1, 2
Spot-check c, ρ, ε	4 windows, 1.25, 10^{-3}
Trust λ, τ, initial T	0.6, 0.3, 0.6

VI. Results and Discussion

A. Proof-of-Presence Accuracy

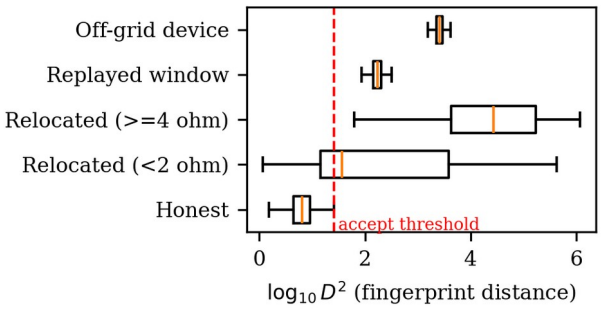


Fig. 3. Distribution of the fingerprint distance D^2 (log scale) for honest nodes and four attacks, Δq = 30 kvar, L = 120 s. The dashed line is the acceptance threshold.

Fig. 3 shows the test statistic over 960 honest node-windows and the attack cases. Honest nodes were accepted in 99.9% of trials, with a median D^2 of 6.5, close to the value 8 expected for a chi-square with eight degrees of freedom. Off-grid devices and replayed windows were never accepted; their median D^2 was 2592 and 173. A genuine device moved to a bus at least 4 Ω away was never accepted either. Moves to an electrically close bus (under 2 Ω) were accepted in 42.4% of cases, which sets the location resolution of the method.

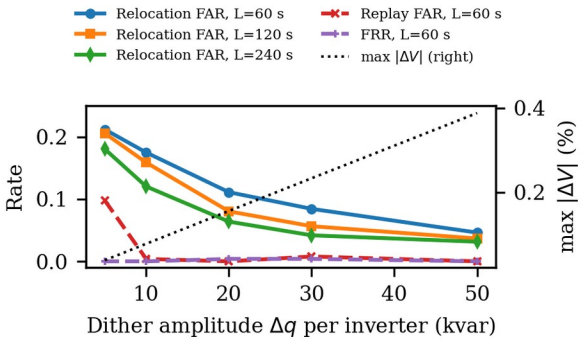


Fig. 4. Relocation false acceptance (FAR) for three window lengths, replay FAR and false rejection (FRR) for 60 s windows, and the largest voltage change caused by the dither (right axis), against dither amplitude.

Fig. 4 shows the trade-off between dither size, window length and grid impact. FRR stayed below 0.6% everywhere. Off-grid and replay acceptance was already zero from 10 kvar with 120 s windows, but at 5 kvar and 60 s some replays got through (9.8%). Relocation FAR over all bus pairs fell from 20.6% at 5 kvar to 5.6% at 30 kvar and 3.2% at 50 kvar with 240 s windows. At 30 kvar the largest voltage change on any bus was only 0.23%, far inside the usual ±6% supply limit, so the challenge is not visible to customers.

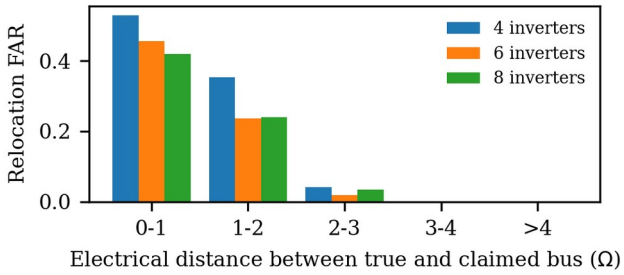


Fig. 5. Relocation false acceptance against the electrical distance between the true and the claimed bus, for 4, 6 and 8 dither inverters.

Adding inverters in the middle of long laterals splits buses that share the same downstream set. As Fig. 5 shows, going from 4 to 8 inverters reduced the FAR for pairs closer than 1 Ω from 53% to 42% and the all-pairs FAR from 5.8% to 4.3%. Beyond 3 Ω no relocation was ever accepted. The location claim is therefore exact at the lateral level and approximate within a few hundred metres on the same lateral.

B. Robustness to Poisoning by Genuine Nodes

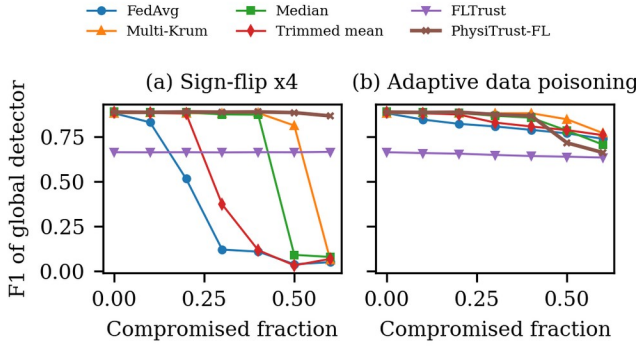


Fig. 6. F1 of the global detector against the fraction of compromised genuine nodes for (a) sign-flip and (b) adaptive data poisoning. Mean of two seeds.

Without attack all methods except FLTrust reached an F1 near 0.88 (AUC 0.954). FLTrust stayed at about 0.66 because its small root set biases the direction of every round. Under sign-flip (Fig. 6a), FedAvg collapsed at 30% attackers (F1 0.12) and the trimmed mean at 40%. Median and Multi-Krum held until 40% and failed at 50% or 60%. PhysiTrust-FL kept F1 = 0.886 at 50% and 0.867 at 60%. The reason is that the provenance gate (9) does not vote: a sign-flipped model has a high loss on the node's own signed windows whatever the other nodes send, so the defence does not need an honest majority.

ADP (Fig. 6b) is the hard case because it is built to pass the loss gate. Its damage is limited by design: even FedAvg only fell to 0.77 at 50%. With 30% attackers PhysiTrust-FL matched the median (0.88 against 0.87). At 50% and 60% it fell to 0.72 and 0.66, below FedAvg and Multi-Krum. Once ADP nodes are half of the passing set, the median reference used for trust follows them and the weighted median amplifies this. This is the main weakness of the scheme and is discussed in Section VII. Table III lists F1 at 30% and 60% for all three attacks.

TABLE III. F1 OF THE GLOBAL DETECTOR WITH 30% AND 60% COMPROMISED NODES

Method	SF 30	SF 60	GN 30	GN 60	ADP 30	ADP 60
FedAvg	0.12	0.05	0.85	0.81	0.81	0.74
Multi-Krum	0.88	0.06	0.88	0.86	0.88	0.77
Median	0.88	0.08	0.89	0.88	0.87	0.71
Trimmed mean	0.37	0.07	0.88	0.85	0.83	0.76
FLTrust	0.66	0.67	0.66	0.66	0.65	0.63
PhysiTrust-FL	0.89	0.87	0.89	0.89	0.88	0.66

C. Sybil Devices with Valid Credentials

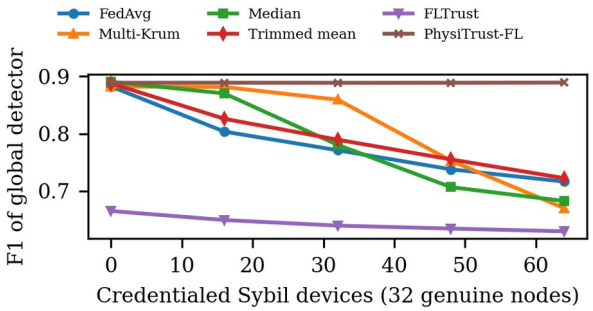


Fig. 7. F1 against the number of credentialed but off-feeder Sybil devices running ADP, with 32 genuine nodes.

Fig. 7 shows the case that identity checks cannot handle: Sybils whose credentials are valid. With 64 Sybils (two for each genuine node), F1 dropped to 0.72 for FedAvg, 0.67 for Multi-Krum, 0.68 for the median and 0.72 for the trimmed mean. PhysiTrust-FL stayed at 0.889, the same as without Sybils, because every Sybil failed the PoP test and was revoked within the first rounds. Adding identities does not help an attacker unless each identity is also wired to the right bus.

D. Ablation

TABLE IV. ABLATION, F1 OF THE GLOBAL DETECTOR

Variant	SF 60%	48 ADP Sybils	ADP 30%
Full PhysiTrust-FL	0.867	0.889	0.876
without PoP	0.867	0.649	0.876
without provenance check	0.029	0.889	0.874
without trust state	0.85	0.888	0.865
weighted mean instead of median	0.877	0.881	0.842

Table IV removes one part at a time. Without PoP, 48 Sybils pulled F1 down to 0.65. Without the provenance gate, the 60% sign-flip case collapsed to 0.03, since the median reference used for trust then belongs to the attackers. Replacing the weighted median with a weighted mean lowered F1 under ADP to 0.84. Each component covers a different threat, which is the argument for claiming them together.

E. Trust and Key-State Dynamics

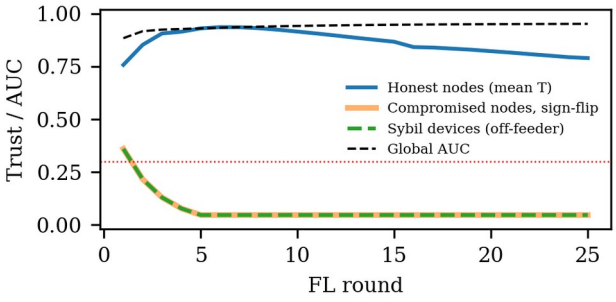


Fig. 8. Mean trust of honest nodes, sign-flip nodes (25%) and 16 Sybils over the rounds, with global AUC. The dotted line is the quarantine threshold.

Fig. 8 follows one run with 25% sign-flip nodes and 16 Sybils. Malicious trust crossed the quarantine line in round 2 and the first revocation was written on chain in round 5. Honest trust rose towards one and the AUC was not disturbed. A node that is revoked cannot come back by rotating its key, because a new key needs a new credential from the utility.

F. Overhead

TABLE V. MEASURED CRYPTO TIME AND ESTIMATED GAS PER CALL

Operation	Cost
SE P-256 sign (window digest)	0.08 ms
SE P-256 verify	0.08 ms
EIP-712 secp256k1 sign	4.77 ms
EIP-712 ecrecover (verify)	6.81 ms
Merkle root of 512 windows	0.48 ms
registerNode (DID, SE key hash, bus, VC hash)	93,848 gas
openRound (seed commit, global hash)	33,524 gas
commitMeasurement (window digest)	45,112 gas
revealSeed (32 B)	30,112 gas
submitUpdate (EIP-712 payload, sig)	73,812 gas
setKeyState (quarantine / revoke)	28,524 gas
anchorModel (CID hash)	28,524 gas

Table V lists the cost per node and round. SE signing of a window took 0.08 ms and EIP-712 signing about 4.8 ms on a laptop CPU in Python. Per round a node sends one commit and one update, about 118k gas in total, mostly from the 21,000 base cost and new storage slots; the 3,000 gas ecrecover precompile is small compared with this [23]. On an L2 or a permissioned EVM chain run by the utility this is negligible. The extra payload over plain FL is 65 bytes of signature and 160 bytes of hashes per update.

VII. NOVELTY, CLAIMS AND LIMITATIONS

The combination that appears new, based on the patent and literature search behind Table I, is the use of an actively injected, ledger-seeded grid perturbation as a per-round credential for FL participants, tied to a hardware-signed data trail and to on-chain key states. A draft claim structure is:

- *Claim 1 (system).* Edge devices with a metering front end, a secure element and a wallet key; a utility actuator applying a perturbation derived from a ledger-committed seed; and a ledger that admits a

model update only if its signature matches an active DID, its measurement commit precedes the seed reveal, and its measured response matches the expected fingerprint of the registered bus.

- *Dependent claims.* Multi-inverter orthogonal chips with voltage and branch-flow fingerprints; the chi-square test with model tolerance; spot-checks of SE-signed windows with a loss gate; the quarantine and revocation machine; trust-weighted median aggregation; dither kept within supply limits.

Against the threats of Section III the scheme gives the following. T1: a credentialed device that is not wired to the feeder sees no dither and fails PoP in every round, so extra identities add no weight. T2: an old window carries the chips of an old seed and an old update reuses a spent nonce, so both are rejected. T3: a moved device shows the fingerprint of its new bus, which is detected beyond about 3 Ω of path impedance. T4: telemetry that did not pass through the SE has no valid signature and cannot be revealed in a spot-check. T5: sign-flip and noise updates fail the loss gate on signed data without needing an honest majority, while adaptive poisoning is bounded by the gate. T6: any change in transit breaks the EIP-712 signature.

The work has limits. Buses closer than about 2 Ω cannot always be told apart. Adaptive poisoning that stays within the loss gate is only bounded, not removed, and once such nodes are half of the genuine set the scheme does worse than plain averaging; a reference built from the utility's own meters is one fix to study. The method needs controllable inverters and an accurate feeder model, and the data here is simulated, so a hardware test on a real feeder is the next step. This paper is not a legal opinion on patentability; a formal search should be done before filing, and filing should come before publication.

VIII. Conclusion

PhysiTrust-FL adds two physical facts to the identity of a federated grid device: that it is wired to the bus it claims right now, and that its training data was measured by its own hardware. On the IEEE 33-bus feeder this blocked off-grid, replayed and distant relocated devices completely, cost less than 0.24% voltage deviation, and kept the global anomaly detector working with 60% sign-flipping nodes or 64 credentialed Sybils. Future work will test the scheme on hardware, deploy the contracts on a test network to measure gas, and study challenge schedules that adapt to the trust of each node.

References

[1] J. Jithish, B. Alangot, N. Mahalingam, and K. S. Yeo, "Distributed anomaly detection in smart grids: A federated learning-based approach," *IEEE Access*, vol. 11, pp. 7157–7179, 2023.

[2] Y. Li et al., "Detection of false data injection attacks in smart grid: A secure federated deep learning approach," *IEEE Trans. Smart Grid*, vol. 13, no. 6, pp. 4862–4872, 2022.

[3] A. S. Musleh, G. Chen, and Z. Y. Dong, "A survey on the detection algorithms for false data injection attacks in smart grids," *IEEE Trans. Smart Grid*, vol. 11, no. 3, pp. 2218–2234, 2020.

[4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.

[5] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017, pp. 119–129.

[6] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. ICML*, 2018, pp. 5650–5659.

[7] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1605–1622.

[8] H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Blockchained on-device federated learning," *IEEE Commun. Lett.*, vol. 24, no. 6, pp. 1279–1283, 2020.

[9] R. Tertulino, R. Almeida, and L. Alencar, "Trustworthy blockchain-based federated learning for electronic health records: Securing participant identity with decentralized identifiers and verifiable credentials," arXiv:2602.02629, 2026.

[10] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022.

[11] Accenture Global Solutions Ltd., "Decentralized federated learning system," U.S. Patent 11 303 448 B2, Apr. 12, 2022.

[12] Beijing Univ. of Posts and Telecommunications, "Blockchain-based asynchronous federated learning method and system," CN Patent 114491623B, Jun. 7, 2024.

[13] Equinix Inc., "Decentralized identifiers for securing device registration and software updates," U.S. Patent 11 379 213 B1, Jul. 5, 2022.

[14] T. Huang, B. Wang, J. Ramos-Ruiz, P. Enjeti, P. R. Kumar, and L. Xie, "Detection of cyber attacks in renewable-rich microgrids using dynamic watermarking," arXiv:2011.12990, 2020.

[15] M. Higgins, F. Teng, and T. Parisini, "Stealthy MTD against unsupervised learning-based blind FDI attacks in power systems," *IEEE Trans. Inf. Forensics Security*, vol. 16, 2021.

[16] D. Nagothu et al., "DeFakePro: Decentralized deepfake attacks detection using ENF authentication," arXiv:2207.13070, 2022.

[17] M. Zhang, C. Sonnadara, S. Shah, and M. Wu, "Feasibility study of location authentication for IoT data using power grid signatures," TechRxiv preprint, 2024.

[18] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. NDSS*, 2021.

[19] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. AISTATS*, 2020, pp. 2938–2948.

[20] P. Braendgaard and J. Torstensson, "ERC-1056: Ethereum lightweight identity," Ethereum Improvement Proposals, 2018.

[21] R. Bloemen, L. Logvinov, and J. Evans, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, 2017.

[22] M. E. Baran and F. F. Wu, "Network reconfiguration in distribution systems for loss reduction and load balancing," *IEEE Trans. Power Del.*, vol. 4, no. 2, pp. 1401–1407, 1989.

[23] C. Patsonakis, K. Samari, A. Kiayias, and M. Roussopoulos, "On the practicality of smart contract PKI," arXiv:1902.00878, 2019.

APPENDIX: SIMULATION CODE (PHYSITRUST_SIM.PY)

Requirements: Python 3.9+, numpy, scipy, matplotlib, eth-account, cryptography. Install with pip install numpy scipy matplotlib eth-account cryptography. Run python physitrust_sim.py for the full experiments (results go to ./results) or add --quick for a short run.

```
"""
PhysiTrust-FL simulation
Grid-challenged proof-of-presence (PoP), hardware-bound measurement provenance
    and DID-bound trust management for federated anomaly detection on the
IEEE 33-bus distribution feeder.

Author : Manojkumar Ashokkumar (VIT Vellore)
  Needs  : Python 3.9+, numpy, scipy, matplotlib, eth-account, cryptography
    Install: pip install numpy scipy matplotlib eth-account cryptography
 Run    : python physitrust_sim.py           (full run, about 15 to 25 min)
          python physitrust_sim.py --quick    (smaller run, about 3 min)
           Output : figures (PNG) and tables (CSV) in ./results
"""
import argparse, hashlib, json, os, time
import numpy as np
from scipy.stats import chi2
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from eth_account import Account
from eth_account.messages import encode_typed_data
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec

OUT = "results"
os.makedirs(OUT, exist_ok=True)
plt.rcParams.update({"font.family": "serif", "font.size": 8,
                     "axes.titlesize": 8, "legend.fontsize": 6.5})

# ---------------------------------------------------------------------------
# 1. IEEE 33-bus feeder (Baran and Wu) and vectorised backward/forward sweep
# ---------------------------------------------------------------------------
BRANCH = np.array([  # from, to, R (ohm), X (ohm)
    [1, 2, .0922, .0470], [2, 3, .4930, .2511], [3, 4, .3660, .1864],
    [4, 5, .3811, .1941], [5, 6, .8190, .7070], [6, 7, .1872, .6188],
    [7, 8, .7114, .2351], [8, 9, 1.030, .7400], [9, 10, 1.044, .7400],
    [10, 11, .1966, .0650], [11, 12, .3744, .1238], [12, 13, 1.468, 1.155],
    [13, 14, .5416, .7129], [14, 15, .5910, .5260], [15, 16, .7463, .5450],
    [16, 17, 1.289, 1.721], [17, 18, .7320, .5740], [2, 19, .1640, .1565],
    [19, 20, 1.5042, 1.3554], [20, 21, .4095, .4784], [21, 22, .7089, .9373],
    [3, 23, .4512, .3083], [23, 24, .8980, .7091], [24, 25, .8960, .7011],
    [6, 26, .2030, .1034], [26, 27, .2842, .1447], [27, 28, 1.059, .9337],
    [28, 29, .8042, .7006], [29, 30, .5075, .2585], [30, 31, .9744, .9630],
    [31, 32, .3105, .3619], [32, 33, .3410, .5302]])
LOAD_KW = np.array([100, 90, 120, 60, 60, 200, 200, 60, 60, 45, 60, 60, 120,
                    60, 60, 60, 90, 90, 90, 90, 90, 420, 420, 60, 60, 60,
                    120, 200, 150, 210, 60], float)
LOAD_KVAR = np.array([60, 40, 80, 30, 20, 100, 100, 20, 20, 30, 35, 35, 80,
                      10, 20, 20, 40, 40, 40, 40, 40, 50, 200, 200, 25, 25,
                      20, 70, 600, 70, 100, 40], float)
VBASE_KV, SBASE_MVA = 12.66, 1.0
ZBASE = VBASE_KV ** 2 / SBASE_MVA
NB, NBR = 33, 32
FB = BRANCH[:, 0].astype(int) - 1
TB = BRANCH[:, 1].astype(int) - 1
ZBR = (BRANCH[:, 2] + 1j * BRANCH[:, 3]) / ZBASE
P_NOM = np.r_[0.0, LOAD_KW / 1000.0]       # per unit on 1 MVA
Q_NOM = np.r_[0.0, LOAD_KVAR / 1000.0]
ACTUATORS = np.array([18, 22, 25, 33]) - 1  # smart-inverter buses (feeder ends)
NODES = np.arange(1, NB)                    # one edge node per load bus 2..33
# branch k ends at bus TB[k] = k+1, so the upstream branch of bus b is b-1


def power_flow(P, Q, iters=30, tol=1e-10):
    """Backward/forward sweep for arrays P, Q of shape (T, 33) in p.u.
    Returns complex bus voltages (T, 33) and receiving-end branch flows."""
    T = P.shape[0]
    V = np.ones((T, NB), complex)
    for _ in range(iters):
        Ibus = np.conj((P + 1j * Q) / V)
        Ibr = np.zeros((T, NBR), complex)
        for k in range(NBR - 1, -1, -1):          # backward sweep
            Ibr[:, k] = Ibus[:, TB[k]]
            Ibus[:, FB[k]] += Ibr[:, k]
        Vn = np.ones((T, NB), complex)
        for k in range(NBR):                      # forward sweep
            Vn[:, TB[k]] = Vn[:, FB[k]] - ZBR[k] * Ibr[:, k]
        done = np.max(np.abs(Vn - V)) < tol
        V = Vn
        if done:
            break
    Sflow = V[:, TB] * np.conj(Ibr)
    return V, Sflow


def electrical_distance():
    """|Z| of the path between every pair of buses (ohm)."""
    parent = {int(TB[k]): int(FB[k]) for k in range(NBR)}
    zb = {int(TB[k]): abs(BRANCH[k, 2] + 1j * BRANCH[k, 3]) for k in range(NBR)}
    def to_root(b):
        p = [b]
        while b in parent:
```

```
                                    b = parent[b]
                                     p.append(b)
                                    return p
                        paths = [to_root(b) for b in range(NB)]
                                D = np.zeros((NB, NB))
                                  for i in range(NB):
                                    for j in range(NB):
                        lca = next(b for b in paths[i] if b in paths[j])
                             up_i = paths[i][:paths[i].index(lca)]
                             up_j = paths[j][:paths[j].index(lca)]
                  D[i, j] = sum(zb[b] for b in up_i) + sum(zb[b] for b in up_j)
                                   return D


# ---------------------------------------------------------------------------
                  # 2. Grid-challenged proof-of-presence (PoP)
# ---------------------------------------------------------------------------
                       def keccak_hex(b: bytes) -> str:
                           from eth_utils import keccak
                              return keccak(b).hex()


    def chips_from_seed(seed: bytes, n_act: int, L: int) -> np.ndarray:
        """Pseudo-random +/-1 spreading codes, one row per actuator."""
                             C = np.zeros((n_act, L))
                              for a in range(n_act):
                                   bits, ctr = [], 0
                                 while len(bits) < L:
        h = hashlib.sha256(seed + bytes([a]) + ctr.to_bytes(4, "big")).digest()
                                      for byte in h:
                                      for s in range(8):
                                   bits.append((byte >> s) & 1)
                                        ctr += 1
                           C[a] = 2 * np.array(bits[:L]) - 1
                                   return C


                                  class Feeder:
  """Generates 1 Hz measurement windows with load noise and optional dither."""
        def __init__(self, rng, sigma_load=0.02, phi=0.95, sigma_v=5e-4,
                             sigma_q_rel=2e-3, sigma_q_abs=2e-4):
                  self.rng, self.sl, self.phi = rng, sigma_load, phi
            self.sv, self.sqr, self.sqa = sigma_v, sigma_q_rel, sigma_q_abs

       def window(self, L, codes=None, dq=0.03, m0=None, acts=ACTUATORS):
                                    rng = self.rng
                 m0 = rng.uniform(0.4, 1.0) if m0 is None else m0
                                 e = np.zeros((L, NB, 2))
                          e[0] = rng.normal(0, self.sl, (NB, 2))
        for t in range(1, L):  # AR(1) load fluctuation, separate for P and Q
               e[t] = self.phi * e[t - 1] + np.sqrt(1 - self.phi ** 2) * \
                                rng.normal(0, self.sl, (NB, 2))
                          P = m0 * P_NOM[None] * (1 + e[..., 0])
                          Q = m0 * Q_NOM[None] * (1 + e[..., 1])
         if codes is not None:  # inverter injects dq*c(t) of reactive power
                                 Q[:, acts] -= dq * codes.T
                                 V, S = power_flow(P, Q)
                 Vm = np.abs(V) + rng.normal(0, self.sv, (L, NB))
                                    Qf = S.imag
      Qf = Qf + rng.normal(0, 1, Qf.shape) * (self.sqr * np.abs(Qf) + self.sqa)
                                head_p = P.sum(1).mean()
                               return Vm, Qf, head_p, m0


                def expected_gains(m0, dq, acts=ACTUATORS):
    """Verifier-side fingerprint: response of |V_b| and upstream Q-flow of
    every bus to +1 chip of each actuator, from the utility feeder model."""
                                    n = len(acts)
                 P = np.repeat((m0 * P_NOM)[None], 1 + 2 * n, 0)
                 Q = np.repeat((m0 * Q_NOM)[None], 1 + 2 * n, 0)
                          for a, bus in enumerate(acts):
                                 Q[1 + 2 * a, bus] -= dq
                                 Q[2 + 2 * a, bus] += dq
                               V, S = power_flow(P, Q)
                              Vm, Qf = np.abs(V), S.imag
                 gV = np.zeros((NB, n)); gQ = np.zeros((NBR, n))
                                  for a in range(n):
                    gV[:, a] = (Vm[1 + 2 * a] - Vm[2 + 2 * a]) / 2
                    gQ[:, a] = (Qf[1 + 2 * a] - Qf[2 + 2 * a]) / 2
                                   return gV, gQ


                        def estimate_gains(y, codes):
      """OLS of a measured series on the codes plus a quadratic trend."""
                                    L = y.shape[0]
                               t = np.linspace(-1, 1, L)
              X = np.column_stack([codes.T, np.ones(L), t, t ** 2])
                   beta, *_ = np.linalg.lstsq(X, y, rcond=None)
                                  res = y - X @ beta
                        s2 = (res ** 2).sum(0) / (L - X.shape[1])
                               XtXi = np.linalg.inv(X.T @ X)
                                    k = codes.shape[0]
       se = np.sqrt(np.outer(s2, np.diag(XtXi)[:k]).T)  # (k, n_series)
                                  return beta[:k], se


        def pop_statistic(Vm_b, Qf_b, codes, gV_b, gQ_b, kappa=0.02):
    """Chi-square distance between measured and expected fingerprint (E5)."""
                    gv_hat, sv = estimate_gains(Vm_b[:, None], codes)
                    gq_hat, sq = estimate_gains(Qf_b[:, None], codes)
                        g_hat = np.r_[gv_hat[:, 0], gq_hat[:, 0]]
```

```
                se = np.r_[sv[:, 0], sq[:, 0]]
                  g_exp = np.r_[gV_b, gQ_b]
              var = se ** 2 + (kappa * np.abs(g_exp)) ** 2
            return float(((g_hat - g_exp) ** 2 / var).sum())


               def pop_threshold(n_act, alpha=1e-3):
                 return chi2.ppf(1 - alpha, 2 * n_act)


            POP_THRESHOLD = pop_threshold(len(ACTUATORS))


                    def m0_from_head(head_p):
                     return head_p / P_NOM.sum()


        def pop_trial(feeder, L, dq, acts=ACTUATORS, rng=None):
    """One challenge window. Returns measured series, codes and expected
  fingerprints, plus an off-grid window and a stale (replayed) window."""
                rng = rng or np.random.default_rng()
           codes = chips_from_seed(rng.bytes(32), len(acts), L)
         Vm, Qf, head_p, _ = feeder.window(L, codes, dq, acts=acts)
           old = chips_from_seed(rng.bytes(32), len(acts), L)
           Vr, Qr, _, _ = feeder.window(L, old, dq, acts=acts)
           Vo, Qo, _, _ = feeder.window(L, None, dq, acts=acts)
          gV, gQ = expected_gains(m0_from_head(head_p), dq, acts)
return dict(codes=codes, Vm=Vm, Qf=Qf, Vr=Vr, Qr=Qr, Vo=Vo, Qo=Qo, gV=gV, gQ=gQ)


# ---------------------------------------------------------------------------
 # 3. Web3 PKI layer: DID keys, secure-element keys, Merkle provenance and a
    #    Python mirror of the on-chain registry / round contract
# ---------------------------------------------------------------------------
      ACTIVE, QUARANTINED, REVOKED = "ACTIVE", "QUARANTINED", "REVOKED"
                          CHAIN_ID = 31337
          CONTRACT = "0x5FbDB2315678afecb367f032d93F642f64180aa3"


                    def sha256(b: bytes) -> bytes:
                     return hashlib.sha256(b).digest()


                       def merkle_root(leaves):
              level = [sha256(x) for x in leaves] or [sha256(b"")]
                           while len(level) > 1:
                               if len(level) % 2:
                               level.append(level[-1])
     level = [sha256(level[i] + level[i + 1]) for i in range(0, len(level), 2)]
                               return level[0]


                        class SecureElement:
     """Emulates a metering secure element (P-256, e.g. ATECC608-class).
     It signs the digest of every raw measurement window it produces."""
                            def __init__(self):
                self._sk = ec.generate_private_key(ec.SECP256R1())
                           self.pub = self._sk.public_key()
                                   self.counter = 0

                   def sign_window(self, raw: np.ndarray):
                                self.counter += 1
                digest = sha256(raw.astype(np.float32).tobytes() +
                                 self.counter.to_bytes(8, "big"))
         return digest, self._sk.sign(digest, ec.ECDSA(hashes.SHA256()))


                    def se_verify(pub, digest, sig):
                                      try:
                pub.verify(sig, digest, ec.ECDSA(hashes.SHA256()))
                                   return True
                               except Exception:
                                   return False


        def typed_update(addr, rnd, ghash, uhash, mroot, nonce):
                                   return {
                                    "types": {
              "EIP712Domain": [{"name": "name", "type": "string"},
                               {"name": "version", "type": "string"},
                              {"name": "chainId", "type": "uint256"},
                          {"name": "verifyingContract", "type": "address"}],
                 "Update": [{"name": "did", "type": "address"},
                            {"name": "round", "type": "uint256"},
                           {"name": "globalHash", "type": "bytes32"},
                           {"name": "updateHash", "type": "bytes32"},
                            {"name": "measRoot", "type": "bytes32"},
                             {"name": "nonce", "type": "uint256"}]},
                             "primaryType": "Update",
            "domain": {"name": "PhysiTrustFL", "version": "1",
                "chainId": CHAIN_ID, "verifyingContract": CONTRACT},
          "message": {"did": addr, "round": rnd, "globalHash": ghash,
                 "updateHash": uhash, "measRoot": mroot, "nonce": nonce}}


                        class EdgeIdentity:
     """DID key (secp256k1, Ethereum account) plus secure-element key."""
                       def __init__(self, bus):
                       self.acct = Account.create()
                      self.addr = self.acct.address
                        self.se = SecureElement()
```

```
                                    self.bus = bus
                                    self.nonce = 0

                        def sign_update(self, rnd, ghash, uhash, mroot):
                                    self.nonce += 1
                         msg = encode_typed_data(full_message=typed_update(
                            self.addr, rnd, ghash, uhash, mroot, self.nonce))
                       return self.nonce, self.acct.sign_message(msg).signature


                                    class Ledger:
                  """Python mirror of the PhysiTrust contract logic (registry, rounds,
               commit-reveal challenge seed, update admission, key state, anchoring)."""
                                    def __init__(self):
                        self.reg, self.state, self.used = {}, {}, set()
                     self.round, self.seed_commit, self.seed = 0, None, None
                  self.global_hash, self.meas_commit, self.log = None, {}, []

                        def register(self, ident: EdgeIdentity, vc_ok=True):
                  if vc_ok:  # utility-issued credential binds DID, SE key and bus
                           self.reg[ident.addr] = (ident.se.pub, ident.bus)
                                    self.state[ident.addr] = ACTIVE

                        def open_round(self, seed_commit, global_hash):
                                    self.round += 1
                        self.seed_commit, self.seed = seed_commit, None
                     self.global_hash, self.meas_commit = global_hash, {}

                        def commit_measurement(self, addr, digest):
                  if self.seed is None:  # commits are only accepted before the reveal
                                    self.meas_commit[addr] = digest
                                             return True
                                          return False

                                 def reveal_seed(self, seed):
                               if keccak_hex(seed) == self.seed_commit:
                                          self.seed = seed
                                             return True
                                          return False

                        def admit(self, addr, nonce, sig, uhash, mroot):
                  """Signature, key state, freshness and challenge-commit checks (E7)."""
                         if addr not in self.reg or self.state[addr] == REVOKED:
                                    return False, "unregistered or revoked"
                                       if (addr, nonce) in self.used:
                                          return False, "replayed nonce"
                                       if addr not in self.meas_commit:
                                    return False, "no challenge commitment"
                        msg = encode_typed_data(full_message=typed_update(
                       addr, self.round, self.global_hash, uhash, mroot, nonce))
                     if Account.recover_message(msg, signature=sig) != addr:
                                       return False, "bad signature"
                                    self.used.add((addr, nonce))
                                         return True, "ok"

                                  def set_state(self, addr, s):
                                    if self.state.get(addr) != s:
                                  self.log.append((self.round, addr, s))
                                          self.state[addr] = s


                                 def benchmark_crypto(n=200):
                  """Timing of the cryptographic operations used per node per round."""
                                    ident = EdgeIdentity(5)
                 raw = np.random.default_rng(0).normal(size=(120, 2)).astype(np.float32)
                                     h32 = "0x" + "11" * 32
                                            res = {}
                                     t = time.perf_counter()
                                        for _ in range(n):
                                   d, s = ident.se.sign_window(raw)
                    res["SE P-256 sign (window digest)"] = (time.perf_counter() - t) / n
                                     t = time.perf_counter()
                                        for _ in range(n):
                                    se_verify(ident.se.pub, d, s)
                           res["SE P-256 verify"] = (time.perf_counter() - t) / n
                                     t = time.perf_counter()
                                        for _ in range(n):
                              nonce, sig = ident.sign_update(1, h32, h32, h32)
                        res["EIP-712 secp256k1 sign"] = (time.perf_counter() - t) / n
         msg = encode_typed_data(full_message=typed_update(ident.addr, 1, h32, h32, h32, nonce))
                                     t = time.perf_counter()
                                        for _ in range(n):
                                 Account.recover_message(msg, signature=sig)
                     res["EIP-712 ecrecover (verify)"] = (time.perf_counter() - t) / n
                              leaves = [os.urandom(32) for _ in range(512)]
                                     t = time.perf_counter()
                                       for _ in range(20):
                                         merkle_root(leaves)
                     res["Merkle root of 512 windows"] = (time.perf_counter() - t) / 20
                        return {k: v * 1e3 for k, v in res.items()}  # milliseconds


                                 def gas_estimate():
                     """Gas per contract call estimated from the EVM fee schedule
                       (21000 base, 16 gas per calldata byte, 22100 new SSTORE,
                      5000 SSTORE update, 3000 ecrecover, about 1500 per event).
                       These are estimates, not measurements on a live chain."""
                     base, cd, new, upd, rec, ev = 21000, 16, 22100, 5000, 3000, 1500
                                          calls = {
        "registerNode (DID, SE key hash, bus, VC hash)": base + 4 * 32 * cd + 3 * new + ev + rec,
               "openRound (seed commit, global hash)": base + 2 * 32 * cd + 2 * upd + ev,
```

```
            "commitMeasurement (window digest)": base + 32 * cd + new + ev,
                "revealSeed (32 B)": base + 32 * cd + upd + 2100 + ev,
"submitUpdate (EIP-712 payload, sig)": base + (6 * 32 + 65) * cd + rec + new + new + ev,
            "setKeyState (quarantine / revoke)": base + 64 * cd + upd + ev,
                 "anchorModel (CID hash)": base + 64 * cd + upd + ev,
                                        }
                                    return calls


 # ---------------------------------------------------------------------------
     # 4. Grid-edge telemetry, anomaly injection and the local autoencoder
 # ---------------------------------------------------------------------------
                  W = 10           # window length (minutes)
                FEAT = 4         # |V|, P, Q, upstream Q-flow


                              def daily_shape(h):
                 return (0.45 + 0.25 * np.exp(-((h - 8.5) / 2.0) ** 2)
                          + 0.45 * np.exp(-((h - 19.5) / 2.5) ** 2))


                        def make_telemetry(rng, days=6):
        """1-minute time series for all buses from repeated power flows."""
                                    T = days * 1440
                         h = (np.arange(T) % 1440) / 60.0
                        shift = rng.uniform(-1.0, 1.0, NB)
            mP = np.stack([daily_shape((h - s) % 24) for s in shift], 1)
                             e = np.zeros((T, NB, 2))
                               for t in range(1, T):
                 e[t] = 0.9 * e[t - 1] + rng.normal(0, 0.02, (NB, 2))
                      P = P_NOM[None] * mP * (1 + e[..., 0])
                      Q = Q_NOM[None] * mP * (1 + e[..., 1])
                                V, S = power_flow(P, Q)
                   Vm = np.abs(V) + rng.normal(0, 5e-4, V.shape)
                     Qf = S.imag + rng.normal(0, 1e-3, S.shape)
                                       data = {}
                                    for b in NODES:
            data[b] = np.stack([Vm[:, b], P[:, b], Q[:, b], Qf[:, b - 1]], 1)
                                     return data


                           def windows(x, stride=5):
                    idx = np.arange(0, x.shape[0] - W + 1, stride)
          return np.stack([x[i:i + W] for i in idx])        # (n, W, FEAT)


                             def inject(win, rng):
        """Cyber anomalies on normalised windows (FDI, ramp, scaling, freeze,
                 noise burst, physically inconsistent P-V move)."""
                                      x = win.copy()
                                kind = rng.integers(6)
                               k = rng.uniform(2.5, 5.0)
                                     if kind == 0:
                               x[:, rng.integers(FEAT)] += k
                                     elif kind == 1:
                              x[:, 1] += np.linspace(0, k, W)
                                     elif kind == 2:
                     x[:, 2] = x[:, 2] * rng.uniform(1.5, 2.5) + 1.0
                                     elif kind == 3:
                        x[:] = x[0] + rng.normal(0, 0.02, x.shape)
                               x[:, rng.integers(FEAT)] += 1.5
                                     elif kind == 4:
                             x += rng.normal(0, 1.5, x.shape)
                                       else:
                                  x[:, 1] += k
                                  x[:, 0] += k
                                  return x


                                 class NodeData:
                             def __init__(self, raw, rng):
                                   n = raw.shape[0]
                 tr, va = raw[: 4 * n // 6], raw[4 * n // 6: 5 * n // 6]
                                te = raw[5 * n // 6:]
                     self.mu, self.sd = tr.mean(0), tr.std(0) + 1e-9
                          f = lambda z: (z - self.mu) / self.sd
                    self.train = windows(f(tr)).reshape(-1, W * FEAT)
                     self.val = windows(f(va)).reshape(-1, W * FEAT)
                                   tw = windows(f(te))
                           lab = rng.random(len(tw)) < 0.5
                              for i in np.where(lab)[0]:
                                tw[i] = inject(tw[i], rng)
        self.test, self.label = tw.reshape(-1, W * FEAT), lab.astype(int)
                    self.fake = np.stack([inject(w, rng) for w in
                         windows(f(tr))[: len(self.train)]]).reshape(-1, W * FEAT)


                               D_IN, H = W * FEAT, 12
                    SHAPES = [(H, D_IN), (H,), (D_IN, H), (D_IN,)]
                    NPARAM = sum(int(np.prod(s)) for s in SHAPES)


                                   def unpack(w):
                                     out, i = [], 0
                                    for s in SHAPES:
         n = int(np.prod(s)); out.append(w[i:i + n].reshape(s)); i += n
                                      return out


                                def ae_scores(w, X):
                               W1, b1, W2, b2 = unpack(w)
```

```
    Z = np.tanh(X @ W1.T + b1)
    return ((Z @ W2.T + b2 - X) ** 2).mean(1)


def local_train(w, X, rng, epochs=1, lr=0.05, bs=32):
    w = w.copy()
    for _ in range(epochs):
        perm = rng.permutation(len(X))
        for s in range(0, len(X), bs):
            xb = X[perm[s:s + bs]]
            W1, b1, W2, b2 = unpack(w)
            A = xb @ W1.T + b1
            Z = np.tanh(A)
            R = Z @ W2.T + b2 - xb
            n = len(xb)
            gR = 2 * R / (n * D_IN)
            gW2, gb2 = gR.T @ Z, gR.sum(0)
            gA = (gR @ W2) * (1 - Z ** 2)
            gW1, gb1 = gA.T @ xb, gA.sum(0)
        g = np.concatenate([gW1.ravel(), gb1, gW2.ravel(), gb2])
        w -= lr * g
    return w


def auc(scores, labels):
    order = np.argsort(scores)
    ranks = np.empty(len(scores)); ranks[order] = np.arange(1, len(scores) + 1)
    pos = labels == 1
    n1, n0 = pos.sum(), (~pos).sum()
    return (ranks[pos].sum() - n1 * (n1 + 1) / 2) / (n1 * n0)


def evaluate(w, nodes):
    s = np.concatenate([ae_scores(w, nd.test) for nd in nodes])
    y = np.concatenate([nd.label for nd in nodes])
    thr = np.percentile(np.concatenate([ae_scores(w, nd.val) for nd in nodes]), 99)
    pred = s > thr
    tp = (pred & (y == 1)).sum(); fp = (pred & (y == 0)).sum()
    fn = (~pred & (y == 1)).sum()
    f1 = 2 * tp / max(2 * tp + fp + fn, 1)
    return auc(s, y), f1


# ---------------------------------------------------------------------------
#  5. Aggregation rules (baselines) and the PhysiTrust-FL round protocol
# ---------------------------------------------------------------------------
def agg_fedavg(U, **k):
    return U.mean(0)


def agg_median(U, **k):
    return np.median(U, 0)


def agg_trimmed(U, beta=0.2, **k):
    n = len(U); t = int(beta * n)
    S = np.sort(U, 0)
    return S[t:n - t].mean(0) if n - 2 * t > 0 else np.median(U, 0)


def agg_multikrum(U, f=0, **k):
    n = len(U)
    f = int(min(f, max((n - 3) // 2, 0)))
    d = ((U[:, None, :] - U[None, :, :]) ** 2).sum(-1)
    score = np.sort(d, 1)[:, 1:n - f - 1].sum(1)
    keep = np.argsort(score)[: max(n - f, 1)]
    return U[keep].mean(0)


def agg_fltrust(U, root=None, **k):
    nr = np.linalg.norm(root) + 1e-12
    cos = U @ root / (np.linalg.norm(U, axis=1) * nr + 1e-12)
    ts = np.maximum(cos, 0)
    if ts.sum() == 0:
        return np.zeros_like(root)
    Un = U * (nr / (np.linalg.norm(U, axis=1, keepdims=True) + 1e-12))
    return (ts[:, None] * Un).sum(0) / ts.sum()


def weighted_median(U, wts):
    """Coordinate-wise weighted median of the rows of U (E9)."""
    idx = np.argsort(U, 0)
    Us = np.take_along_axis(U, idx, 0)
    cw = np.cumsum(wts[idx], 0)
    k = (cw >= 0.5 * wts.sum()).argmax(0)
    return Us[k, np.arange(U.shape[1])]


BASELINES = {"FedAvg": agg_fedavg, "Multi-Krum": agg_multikrum,
             "Median": agg_median, "Trimmed mean": agg_trimmed,
             "FLTrust": agg_fltrust}


class World:
    """Telemetry, node datasets, identities and SE-signed training windows
    for one random seed (built once, reused by every method)."""
    def __init__(self, seed):
        self.rng = np.random.default_rng(seed)
        tel = make_telemetry(self.rng)
        self.nodes = [NodeData(tel[b], self.rng) for b in NODES]
        self.ids = [EdgeIdentity(b) for b in NODES]
```

```
                                self.signed = []
                    for nd, idt in zip(self.nodes, self.ids):
              self.signed.append([idt.se.sign_window(x) for x in nd.train])
              pick = self.rng.choice(len(self.nodes), 3, replace=False)
       self.root = np.concatenate([self.nodes[i].train[:40] for i in pick])
                    self.w0 = self.rng.normal(0, 0.1, NPARAM)


          def malicious_update(attack, w, nd, rng, gamma=4.0):
  if attack == "DP":       # trains on fabricated anomalies labelled normal
                  return local_train(w, nd.fake, rng, epochs=3) - w
   if attack == "ADP":      # adaptive: keeps its loss on real data low so
     n = len(nd.train) // 2   # that a spot-check on genuine windows passes
                 X = np.concatenate([nd.fake[:n], nd.train[:n]])
                     u = local_train(w, X, rng, epochs=2) - w
                     l0 = ae_scores(w, nd.train[:200]).mean()
                                for _ in range(10):
              if ae_scores(w + u, nd.train[:200]).mean() <= 1.15 * l0:
                                        break
                                    u = 0.5 * u
                                   return u
                 honest = local_train(w, nd.train, rng) - w
              if attack == "SF":       # sign flip with scaling
                              return -gamma * honest
           if attack == "GN":       # Gaussian noise of large norm
return rng.normal(0, 1, NPARAM) * (5 * np.linalg.norm(honest) / np.sqrt(NPARAM))
                                return honest


 def run_fl(world, method, attack="SF", frac=0.0, n_sybil=0, rounds=30,
                                sybil_attack="ADP",
                seed=0, ablate=(), L=120, dq=0.03, c_spot=4, lam=0.6,
                            tau_q=0.3, track=False):
                   rng = np.random.default_rng(1000 + seed)
                   nodes, N = world.nodes, len(world.nodes)
   bad = set(rng.choice(N, int(round(frac * N)), replace=False).tolist())
             feeder = Feeder(np.random.default_rng(2000 + seed))
                             w = world.w0.copy()
                     proposed = method == "PhysiTrust-FL"
                                if proposed:
                                ledger = Ledger()
   syb_ids = [EdgeIdentity(int(rng.integers(1, NB))) for _ in range(n_sybil)]
                         for idt in world.ids + syb_ids:
        ledger.register(idt)          # Sybil devices hold valid credentials
                    T = {i: 0.6 for i in range(N + n_sybil)}
          streak_q = {i: 0 for i in T}; streak_ok = {i: 0 for i in T}
   hist, stats = [], {"pop_reject": 0, "prov_reject": 0, "sig_reject": 0}
                          for r in range(rounds):
     # ---- local updates ------------------------------------------------
                                    U = []
                           for i, nd in enumerate(nodes):
              U.append(malicious_update(attack, w, nd, rng) if i in bad
                               else local_train(w, nd.train, rng) - w)
          for j in range(n_sybil):  # Sybils hold copied data and attack
                           nd = nodes[int(rng.integers(N))]
                  U.append(malicious_update(sybil_attack, w, nd, rng))
                                U = np.array(U)
                               if not proposed:
                              f = len(bad) + n_sybil
     root = local_train(w, world.root, rng) - w if method == "FLTrust" else None
                     w = w + BASELINES[method](U, f=f, root=root)
                                    if track:
                               hist.append(evaluate(w, nodes)[0])
                                    continue
     # ---- PhysiTrust-FL: challenge, commit, reveal, admit, verify ---------
                                seed_b = rng.bytes(32)
                    ghash = "0x" + sha256(w.tobytes()).hex()
                  ledger.open_round(keccak_hex(seed_b), ghash)
               codes = chips_from_seed(seed_b, len(ACTUATORS), L)
     Vm, Qf, head_p, _ = feeder.window(L, codes, dq)          # real feeder
    Vo, Qo, _, _ = feeder.window(L, None, dq)                # off-grid site
                            all_ids = world.ids + syb_ids
       for idt in all_ids:   # measurement digests committed before reveal
                   ledger.commit_measurement(idt.addr, b"digest")
                            ledger.reveal_seed(seed_b)
                gV, gQ = expected_gains(m0_from_head(head_p), dq)
       checks = []                          # pass 1: PKI, PoP and provenance
                         for i, idt in enumerate(all_ids):
                         if ledger.state[idt.addr] == REVOKED:
                                        continue
                     uhash = "0x" + sha256(U[i].tobytes()).hex()
     mroot = "0x" + merkle_root([d for d, _ in world.signed[i % N][:64]]).hex()
           nonce, sig = idt.sign_update(ledger.round, ghash, uhash, mroot)
               ok, _ = ledger.admit(idt.addr, nonce, sig, uhash, mroot)
                                    if not ok:
                                stats["sig_reject"] += 1
                                        continue
                                    b = idt.bus
                                     if i < N:
           st = pop_statistic(Vm[:, b], Qf[:, b - 1], codes, gV[b], gQ[b - 1])
                 else:          # Sybil device is not on this feeder
           st = pop_statistic(Vo[:, b], Qo[:, b - 1], codes, gV[b], gQ[b - 1])
                   pop_ok = ("pop" in ablate) or st < POP_THRESHOLD
                                   prov_ok = True
          if "prov" not in ablate:   # spot-check SE-signed training windows
                                        k = i % N
               sel = rng.choice(len(nodes[k].train), c_spot, replace=False)
                            pub = idt.se.pub if i < N else None
          good = i < N and all(se_verify(pub, *world.signed[k][s]) for s in sel)
              if i >= N:   # Sybil device signs its own (off-feeder) windows
                                        good = True
```

```
                                    Xs = nodes[k].train[sel]
                                l_new = ae_scores(w + U[i], Xs).mean()
                                    l_old = ae_scores(w, Xs).mean()
                            prov_ok = good and l_new <= 1.25 * l_old + 1e-3
                               stats["pop_reject"] += int(not pop_ok)
                         stats["prov_reject"] += int(pop_ok and not prov_ok)
                              checks.append((i, idt, pop_ok, prov_ok))
                        passed = [i for i, _, p, q in checks if p and q]
                ref = np.median(U[passed], 0) if passed else np.zeros(NPARAM)
            Uagg, Wagg = [], []              # pass 2: trust, key state, weights
                           for i, idt, pop_ok, prov_ok in checks:
          cosv = float(U[i] @ ref / (np.linalg.norm(U[i]) * np.linalg.norm(ref) + 1e-12))
                    s_i = float(pop_ok) * float(prov_ok) * max(0.0, cosv)
                                        if "trust" in ablate:
                                          if pop_ok and prov_ok:
                                        Uagg.append(U[i]); Wagg.append(1.0)
                                                continue
                                    T[i] = lam * T[i] + (1 - lam) * s_i
                                       stt = ledger.state[idt.addr]
                                     if stt == ACTIVE and T[i] < tau_q:
                         ledger.set_state(idt.addr, QUARANTINED); streak_q[i] = 0
                                      elif stt == QUARANTINED:
                                            streak_q[i] += 1
                          streak_ok[i] = streak_ok[i] + 1 if s_i >= 0.6 else 0
                                       if streak_ok[i] >= 2:
                        ledger.set_state(idt.addr, ACTIVE); T[i] = max(T[i], tau_q + 0.05)
                                        elif streak_q[i] >= 3:
                                    ledger.set_state(idt.addr, REVOKED)
                    if ledger.state[idt.addr] == ACTIVE and pop_ok and prov_ok:
                                        Uagg.append(U[i]); Wagg.append(T[i])
                                           if Uagg:
                                    Ua, Wa = np.array(Uagg), np.array(Wagg)
                                cap = np.median(np.linalg.norm(Ua, axis=1))
          Ua = Ua * np.minimum(1, cap / (np.linalg.norm(Ua, axis=1, keepdims=True) + 1e-12))
                                        if Wa.sum() > 0:
                       if "wmedian" in ablate:      # plain trust-weighted mean
                                w = w + (Wa[:, None] * Ua).sum(0) / Wa.sum()
                    else:                           # trust-weighted coordinate median
                                    w = w + weighted_median(Ua, Wa)
                                         if track:
                                    hist.append((evaluate(w, nodes)[0],
                                   np.mean([T[i] for i in range(N) if i not in bad]),
                                    np.mean([T[i] for i in bad]) if bad else np.nan,
                     np.mean([T[i] for i in range(N, N + n_sybil)]) if n_sybil else np.nan))
                                     a, f1 = evaluate(w, nodes)
                 out = {"auc": a, "f1": f1, "hist": hist, "stats": stats}
                                        if proposed:
            out["revoked"] = sum(1 for v in ledger.state.values() if v == REVOKED)
                                     out["log"] = ledger.log
                                          return out


# ---------------------------------------------------------------------------
                        # 6. Experiments, figures and tables
# ---------------------------------------------------------------------------
   METHODS = ["FedAvg", "Multi-Krum", "Median", "Trimmed mean", "FLTrust",
                                       "PhysiTrust-FL"]
                    COL = 3.45  # IEEE column width in inches


                           def save_csv(name, header, rows):
                                       import csv
                 with open(os.path.join(OUT, name), "w", newline="") as f:
                                     wr = csv.writer(f)
                                    wr.writerow(header)
                                       for r in rows:
                                  wr.writerow([str(x) for x in r])


                       def exp_pop_distribution(n_win=30, seed=11):
                              rng = np.random.default_rng(seed)
                                     fd = Feeder(rng)
                                  D = electrical_distance()
        out = {"Honest": [], "Relocated (<2 ohm)": [], "Relocated (>=4 ohm)": [],
                           "Replayed window": [], "Off-grid device": []}
                                    for _ in range(n_win):
                              tr = pop_trial(fd, 120, 0.03, rng=rng)
                           c, gV, gQ = tr["codes"], tr["gV"], tr["gQ"]
                                         for b in NODES:
    out["Honest"].append(pop_statistic(tr["Vm"][:, b], tr["Qf"][:, b - 1], c, gV[b], gQ[b - 1]))
out["Replayed window"].append(pop_statistic(tr["Vr"][:, b], tr["Qr"][:, b - 1], c, gV[b], gQ[b - 1]))
out["Off-grid device"].append(pop_statistic(tr["Vo"][:, b], tr["Qo"][:, b - 1], c, gV[b], gQ[b - 1]))
                                        for j in NODES:
                                             if j == b:
                                                continue
             st = pop_statistic(tr["Vm"][:, j], tr["Qf"][:, j - 1], c, gV[b], gQ[b - 1])
                                          if D[b, j] < 2:
                                    out["Relocated (<2 ohm)"].append(st)
                                       elif D[b, j] >= 4:
                                    out["Relocated (>=4 ohm)"].append(st)
                                          rows = []
                                    for k, v in out.items():
                                          v = np.array(v)
              rows.append([k, len(v), np.mean(v < POP_THRESHOLD), np.median(v)])
  save_csv("table_pop_acceptance.csv", ["case", "trials", "accept_rate", "median_D2"], rows)
                        fig, ax = plt.subplots(figsize=(COL, 1.9))
                  data = [np.log10(np.array(v) + 1e-3) for v in out.values()]
                   ax.boxplot(data, vert=False, widths=0.6, showfliers=False)
                             ax.set_yticks(range(1, len(out) + 1))
                              ax.set_yticklabels(list(out.keys()))
              ax.axvline(np.log10(POP_THRESHOLD), color="r", ls="--", lw=1)
```

```
            ax.text(np.log10(POP_THRESHOLD) + 0.08, 0.55, "accept threshold",
                                    color="r", fontsize=6.5)
                ax.set_xlabel(r"$\log_{10} D^2$ (fingerprint distance)")
fig.tight_layout(); fig.savefig(os.path.join(OUT, "fig_pop_distribution.png"), dpi=300)
                                    plt.close(fig)
                                      return rows


                              def max_voltage_impact(dq):
          P = np.repeat(P_NOM[None], 2, 0); Q = np.repeat(Q_NOM[None], 2, 0)
                                  Q[1, ACTUATORS] -= dq
                                 V, _ = power_flow(P, Q)
                return 100 * np.max(np.abs(np.abs(V[1]) - np.abs(V[0])))


    def exp_pop_sweep(n_win=8, seed=12, dqs=(0.005, 0.01, 0.02, 0.03, 0.05),
                                        Ls=(60, 120, 240)):
                               rng = np.random.default_rng(seed)
                                       fd = Feeder(rng)
                                          rows = []
                                        for L in Ls:
                                         for dq in dqs:
                               frr, far_o, far_r, far_l = [], [], [], []
                                        for _ in range(n_win):
                                     tr = pop_trial(fd, L, dq, rng=rng)
                                c, gV, gQ = tr["codes"], tr["gV"], tr["gQ"]
                                                for b in NODES:
 frr.append(pop_statistic(tr["Vm"][:, b], tr["Qf"][:, b - 1], c, gV[b], gQ[b - 1]) >= POP_THRESHOLD)
 far_o.append(pop_statistic(tr["Vo"][:, b], tr["Qo"][:, b - 1], c, gV[b], gQ[b - 1]) < POP_THRESHOLD)
 far_r.append(pop_statistic(tr["Vr"][:, b], tr["Qr"][:, b - 1], c, gV[b], gQ[b - 1]) < POP_THRESHOLD)
                                        for j in NODES[::3]:
                                            if j != b:
    far_l.append(pop_statistic(tr["Vm"][:, j], tr["Qf"][:, j - 1], c, gV[b], gQ[b - 1]) < POP_THRESHOLD)
              rows.append([L, round(dq * 1000), np.mean(frr), np.mean(far_o),
                           np.mean(far_r), np.mean(far_l), max_voltage_impact(dq)])
      save_csv("table_pop_sweep.csv", ["L_s", "dq_kvar", "FRR", "FAR_offgrid",
                                "FAR_replay", "FAR_relocation", "max_dV_pct"], rows)
                                   plot_pop_sweep(rows)
                                       return rows


                    def plot_pop_sweep(rows, Ls=(60, 120, 240)):
                                R = np.array(rows, float)
                       fig, ax = plt.subplots(figsize=(COL, 2.0))
                              for L, mk in zip(Ls, "osd"):
                                    m = R[:, 0] == L
      ax.plot(R[m, 1], R[m, 5], marker=mk, ms=3, label=f"Relocation FAR, L={L} s")
                                    m = R[:, 0] == Ls[0]
ax.plot(R[m, 1], R[m, 4], marker="x", ms=3, ls="--", label=f"Replay FAR, L={Ls[0]} s")
   ax.plot(R[m, 1], R[m, 2], marker="+", ms=3, ls="--", label=f"FRR, L={Ls[0]} s")
          ax.set_xlabel(r"Dither amplitude $\Delta q$ per inverter (kvar)")
                    ax.set_ylabel("Rate"); ax.set_ylim(-0.01, 0.25)
                                    ax2 = ax.twinx()
ax2.plot(R[m, 1], R[m, 6], color="k", lw=1, ls=":", label=r"max $|\Delta V|$ (right)")
                        ax2.set_ylabel(r"max $|\Delta V|$ (%)")
  h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
fig.legend(h1 + h2, l1 + l2, ncol=2, loc="upper center", fontsize=5.5, frameon=False)
                        fig.tight_layout(rect=(0, 0, 1, 0.78))
             fig.savefig(os.path.join(OUT, "fig_pop_sweep.png"), dpi=300)
                                    plt.close(fig)


                     def exp_pop_actuators(n_win=6, seed=13):
                           rng = np.random.default_rng(seed)
                                   fd = Feeder(rng)
                                D = electrical_distance()
  sets = {"4 inverters": [18, 22, 25, 33], "6 inverters": [18, 22, 25, 33, 13, 29],
                       "8 inverters": [18, 22, 25, 33, 9, 13, 29, 4]}
                  bins = [(0, 1), (1, 2), (2, 3), (3, 4), (4, 100)]
                                   rows, curves = [], {}
                             for name, acts in sets.items():
                                  acts = np.array(acts) - 1
                                th = pop_threshold(len(acts))
                               acc = np.zeros((NB, NB)); frr = []
                                     for _ in range(n_win):
                        tr = pop_trial(fd, 120, 0.03, acts=acts, rng=rng)
                           c, gV, gQ = tr["codes"], tr["gV"], tr["gQ"]
                                        for b in NODES:
    frr.append(pop_statistic(tr["Vm"][:, b], tr["Qf"][:, b - 1], c, gV[b], gQ[b - 1]) >= th)
                                        for j in NODES:
                                            if j != b:
        acc[b, j] += pop_statistic(tr["Vm"][:, j], tr["Qf"][:, j - 1], c, gV[b], gQ[b - 1]) < th
                                     acc /= n_win
    pairs = np.array([(D[b, j], acc[b, j]) for b in NODES for j in NODES if j != b])
 curve = [pairs[(pairs[:, 0] >= lo) & (pairs[:, 0] < hi), 1].mean() for lo, hi in bins]
                                   curves[name] = curve
  rows.append([name, np.mean(frr), pairs[:, 1].mean()] + [round(x, 4) for x in curve])
       save_csv("table_pop_actuators.csv", ["set", "FRR", "FAR_all_pairs"] +
                        [f"FAR_{lo}-{hi}ohm" for lo, hi in bins], rows)
                      fig, ax = plt.subplots(figsize=(COL, 1.7))
                        x = np.arange(len(bins)); wbar = 0.27
                    for k, (name, cv) in enumerate(curves.items()):
                      ax.bar(x + (k - 1) * wbar, cv, wbar, label=name)
     ax.set_xticks(x); ax.set_xticklabels(["0-1", "1-2", "2-3", "3-4", ">4"])
   ax.set_xlabel(r"Electrical distance between true and claimed bus ($\Omega$)")
            ax.set_ylabel("Relocation FAR"); ax.legend(frameon=False)
fig.tight_layout(); fig.savefig(os.path.join(OUT, "fig_pop_actuators.png"), dpi=300)
                                    plt.close(fig)
                                      return rows
```

```
def exp_fl_sweep(worlds, rounds, fracs=(0, .1, .2, .3, .4, .5, .6),
                                 attacks=("SF", "GN", "ADP")):
                                     rows = []
                                 for att in attacks:
                                     for fr in fracs:
                                       for m in METHODS:
                         res = [run_fl(wd, m, att, fr, 0, rounds, seed=s)
                                        for s, wd in enumerate(worlds)]
                    rows.append([att, fr, m, np.mean([r["auc"] for r in res]),
                                         np.mean([r["f1"] for r in res])])
 print(f"  {att} f={fr:.1f} {m:14s} AUC={rows[-1][3]:.3f} F1={rows[-1][4]:.3f}", flush=True)
                         if set(attacks) >= {"SF", "ADP"}:
 save_csv("table_fl_sweep.csv", ["attack", "fraction", "method", "AUC", "F1"], rows)
                                 plot_fl_sweep(rows)
                                   return rows


                def plot_fl_sweep(rows, attacks=("SF", "ADP")):
             names = {"SF": "(a) Sign-flip x4", "GN": "Gaussian noise",
                              "ADP": "(b) Adaptive data poisoning"}
    fig, axs = plt.subplots(1, len(attacks), figsize=(COL, 2.0), sharey=True)
                         for ax, att in zip(axs, attacks):
                          for m, mk in zip(METHODS, "o^sdvx"):
      pts = [(float(r[1]), float(r[4])) for r in rows if r[0] == att and r[2] == m]
                           ax.plot(*zip(*pts), marker=mk, ms=2.5,
                         lw=1.4 if m == "PhysiTrust-FL" else 0.8, label=m)
           ax.set_title(names[att]); ax.set_xlabel("Compromised fraction")
                   axs[0].set_ylabel("F1 of global detector")
                    h, l = axs[0].get_legend_handles_labels()
    fig.legend(h, l, loc="upper center", ncol=3, frameon=False, fontsize=5.5)
                         fig.tight_layout(rect=(0, 0, 1, 0.84))
           fig.savefig(os.path.join(OUT, "fig_fl_sweep.png"), dpi=300)
                                 plt.close(fig)


     def exp_sybil(worlds, rounds, counts=(0, 16, 32, 48, 64), tag=""):
                                     rows = []
                                 for n in counts:
                                   for m in METHODS:
   res = [run_fl(wd, m, "SF", 0.0, n, rounds, seed=s) for s, wd in enumerate(worlds)]
 rows.append([n, m, np.mean([r["auc"] for r in res]), np.mean([r["f1"] for r in res])])
  print(f"  sybil={n} {m:14s} AUC={rows[-1][2]:.3f} F1={rows[-1][3]:.3f}", flush=True)
   save_csv(f"table_sybil{tag}.csv", ["sybils", "method", "AUC", "F1"], rows)
                                     if tag:
                                     return rows
                                 plot_sybil(rows)
                                   return rows


                                 def plot_sybil(rows):
                         fig, ax = plt.subplots(figsize=(COL, 1.8))
                            for m, mk in zip(METHODS, "o^sdvx"):
                  pts = [(int(r[0]), float(r[3])) for r in rows if r[1] == m]
                           ax.plot(*zip(*pts), marker=mk, ms=3, label=m)
              ax.set_xlabel("Credentialed Sybil devices (32 genuine nodes)")
                             ax.set_ylabel("F1 of global detector")
fig.legend(*ax.get_legend_handles_labels(), loc="upper center", ncol=3, frameon=False, fontsize=5.5)
  fig.tight_layout(rect=(0, 0, 1, 0.84)); fig.savefig(os.path.join(OUT, "fig_sybil.png"), dpi=300)
                                 plt.close(fig)
                                   return rows


                 def exp_ablation(worlds, rounds, which=None, tag=""):
                cases = [("Full PhysiTrust-FL", ()), ("without PoP", ("pop",)),
             ("without provenance check", ("prov",)), ("without trust state", ("trust",)),
                      ("weighted mean instead of median", ("wmedian",))]
                scen = [("SF", 0.6, 0), ("ADP", 0.0, 48), ("ADP", 0.3, 0)]
                                     rows = []
                                for ci, (name, ab) in enumerate(cases):
                                 if which is not None and ci not in which:
                                             continue
                                       row = [name]
                                for att, fr, ns in scen:
          res = [run_fl(wd, "PhysiTrust-FL", att, fr, ns, rounds, seed=s, ablate=ab)
                                        for s, wd in enumerate(worlds)]
                    row.append(round(np.mean([r["f1"] for r in res]), 3))
                                       rows.append(row)
                             print("  ablation", row, flush=True)
 save_csv(f"table_ablation{tag}.csv", ["variant", "F1_SF60", "F1_ADPsybil48",
                                                 "F1_ADP30"], rows)
                                   return rows


                             def exp_trust(world, rounds):
 r = run_fl(world, "PhysiTrust-FL", "SF", 0.25, 16, rounds, seed=0, track=True)
                                 H = np.array(r["hist"], float)
                         fig, ax = plt.subplots(figsize=(COL, 1.8))
                                   x = np.arange(1, len(H) + 1)
                    ax.plot(x, H[:, 1], label="Honest nodes (mean T)")
  ax.plot(x, H[:, 2], lw=2.2, alpha=0.6, label="Compromised nodes, sign-flip")
        ax.plot(x, H[:, 3], ls="--", label="Sybil devices (off-feeder)")
             ax.plot(x, H[:, 0], "k--", lw=0.9, label="Global AUC")
                    ax.axhline(0.3, color="r", lw=0.6, ls=":")
ax.set_xlabel("FL round"); ax.set_ylabel("Trust / AUC"); ax.set_ylim(-0.02, 1.02)
            ax.legend(frameon=False, fontsize=5.5, loc="center right")
  fig.tight_layout(); fig.savefig(os.path.join(OUT, "fig_trust.png"), dpi=300)
                                 plt.close(fig)
                                   first = {}
                               for rnd, addr, st in r["log"]:
                                   first.setdefault(st, rnd)
save_csv("table_trust_events.csv", ["state", "first_round"], list(first.items()))
```

```
return r


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true", help="1 seed, 15 rounds")
    ap.add_argument("--stage", default="all",
                    choices=["all", "crypto", "pop", "fl", "sybil", "ablation", "trust"],
                    help="run one stage only (useful on slow machines)")
    ap.add_argument("--attack", default=None, help="fl stage: SF, GN or ADP only")
    ap.add_argument("--fracs", default=None, help="fl stage: e.g. 0,0.1,0.2")
    ap.add_argument("--counts", default=None,
                    help="sybil stage: Sybil counts, e.g. 0,16; ablation stage: variant indices, e.g. 0,1")
    ap.add_argument("--tag", default="", help="suffix for CSV names of partial runs")
    ap.add_argument("--replot", action="store_true",
                    help="redraw the FL sweep figure from the saved CSV files")
    a = ap.parse_args()
    seeds = [0] if a.quick else [0, 1]
    rounds = 15 if a.quick else 25
    st = a.stage
    t0 = time.time()
    if a.replot:
        import csv, glob
        rows = []
        for fn in sorted(glob.glob(os.path.join(OUT, "table_fl_sweep_*.csv"))):
            with open(fn) as f:
                rows += list(csv.reader(f))[1:]
        plot_fl_sweep(rows)
        with open(os.path.join(OUT, "table_pop_sweep.csv")) as f:
            plot_pop_sweep(list(csv.reader(f))[1:])
        srows = []
        for fn in sorted(glob.glob(os.path.join(OUT, "table_sybil_*.csv"))):
            with open(fn) as f:
                srows += list(csv.reader(f))[1:]
        if srows:
            srows.sort(key=lambda r: int(r[0]))
            save_csv("table_sybil.csv", ["sybils", "method", "AUC", "F1"], srows)
            plot_sybil(srows)
        arows = []
        for fn in sorted(glob.glob(os.path.join(OUT, "table_ablation_*.csv"))):
            with open(fn) as f:
                arows += list(csv.reader(f))[1:]
        if arows:
            save_csv("table_ablation.csv", ["variant", "F1_SF60", "F1_ADPsybil48", "F1_ADP30"], arows)
        return
    if st in ("all", "crypto"):
        cb = benchmark_crypto(50 if a.quick else 200)
        for k, v in cb.items():
            print(f"  {k:32s} {v:8.3f} ms")
        save_csv("table_crypto_ms.csv", ["operation", "ms"], [[k, round(v, 4)] for k, v in cb.items()])
        save_csv("table_gas_estimate.csv", ["call", "gas"], list(gas_estimate().items()))
    if st in ("all", "pop"):
        print(exp_pop_distribution(10 if a.quick else 30))
        print(exp_pop_sweep(3 if a.quick else 8))
        print(exp_pop_actuators(2 if a.quick else 6))
    if st in ("all", "fl", "sybil", "ablation", "trust"):
        worlds = [World(s) for s in seeds]
        if st in ("all", "fl"):
            atts = (a.attack,) if a.attack else ("SF", "GN", "ADP")
            rows = []
            fr = tuple(float(x) for x in a.fracs.split(",")) if a.fracs else (0, .1, .2, .3, .4, .5, .6)
            for att in atts:
                rows += exp_fl_sweep(worlds, rounds, fracs=fr, attacks=(att,))
                save_csv(f"table_fl_sweep_{att}{a.tag}.csv",
                         ["attack", "fraction", "method", "AUC", "F1"],
                         [r for r in rows if r[0] == att])
        if st in ("all", "sybil"):
            cn = tuple(int(x) for x in a.counts.split(",")) if a.counts else (0, 16, 32, 48, 64)
            exp_sybil(worlds, rounds, counts=cn, tag=a.tag)
        if st in ("all", "ablation"):
            wh = [int(x) for x in a.counts.split(",")] if a.counts else None
            exp_ablation(worlds, rounds, which=wh, tag=a.tag)
        if st in ("all", "trust"):
            exp_trust(worlds[0], rounds)
    print(f"Done in {(time.time() - t0) / 60:.1f} min. Results in ./{OUT}")


if __name__ == "__main__":
    main()
```